



WEB SECURITY



CHAPTER 2: SQL INJECTION

Le Thi Minh Chau
Faculty Of Information Technology
HCMC University Of Technology And Education

Outline



2

1. Introduction
2. What is SQL Injection?
3. SQL Injection Attack
4. Scenario
5. SQL Injection Countermeasures

Introduction



3

- SQL Injection Attacks are ranked as one of the most common form of attack on web applications (Common Vulnerabilities and Exposures list – CVE <http://cve.mitre.org/>).
- Injection are mentioned as the first security risk in Web Application (OWASP Top 10 – 2017).

Introduction



4

- CardSystems, credit card payment processing ruined by SQL Injection Attack in June 2005
 - ▣ 263,000 credit cards were stolen from its DB.
 - ▣ Credit cards were stored unencrypted, 40 million were exposed.



What is SQL Injection?

5

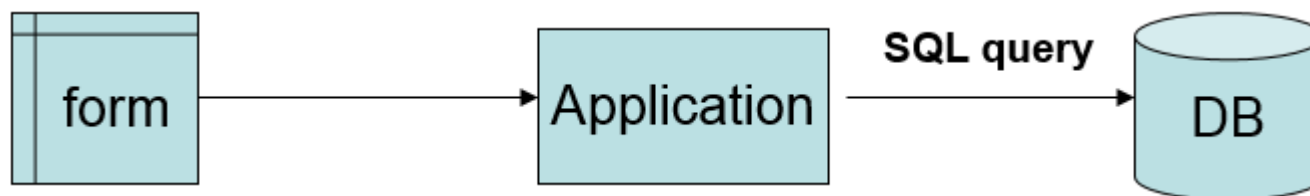
- ❑ SQL Injection is a technique that exploits a security vulnerability occurring in the database layer of an application.
- ❑ A SQL Injection Attack involves the alteration of SQL statements issued by the application.



What is SQL Injection?

6

- The vulnerability occurs when:
 - ▣ The application does not check the user input
 - ▣ Incorrectly filters user input for string literal escape characters embedded in SQL statements
 - ▣ When user input is not strongly typed.





What is SQL Injection?

7

- Injection Mechanisms:
 - ▣ Through the user input (e.g., HTTP GET/POST parameters).
 - ▣ Through cookies:
 - If a Web Application uses the cookie's contents to build SQL queries, an attacker could easily submit an attack by embedding the “malicious” input in the cookie.

SQL Injection Attack



8

□ Attack Objectives:

1. **Identify application injectable parameters:** discover which parameters and user-input fields are vulnerable to SQL Injection.
2. **Performing database finger-printing:** discover the type and version of database that a Web application is using
3. **Determine the database schema:** know database schema information such as table names, column names and column data types to correctly extract data from a database.

SQL Injection Attack



9

4. **Extracting data**: employ techniques that will extract data values from the database. Depending on the type of the Web Application, this information could be sensitive and valuable to the attacker.
5. **Adding or modifying data**
6. **Performing Denial of Service**: lock or drop database tables, shut down the database, thus denying service to other users.
7. **Evading detection**: employ certain attack technique to avoid auditing and detection by system protection mechanisms.

SQL Injection Attack



10

8. **Bypassing Authentication**: assume the rights and privileges associated with another application users.
9. **Executing remote commands**: attempt to execute arbitrary commands on the database. These commands can be stored procedures or functions available to database users.
10. **Performing privilege escalation**: take advantage of implementation errors or logical flaws in the database in order to escalate the privileges of the attacker. As opposed to bypassing authentication attacks, these attacks focus on exploiting the database user privileges.

SQL Injection Attack



11

□ Attack Phases:

1. Exploring
2. Experiment
3. Try to exploit the vulnerabilities discovered.

SQL Injection Attack – Explore



12

1. **Explore**: determine the functionalities exposed by the application
 - By sniffing and analyzing the traffic to/from a given Web service.
 - By examining the landing HTML page of the website.

SQL Injection Attack – Experiment

13

2. **Experiment:** determine user-controllable input susceptible to injection.
 - Inject characters that have special meaning in SQL such as ' " = (
 - Inject input through text fields or through HTTP GET parameters.
 - Modify HTTP POST parameters, hidden fields by using a Web application debugging tool.
 - Analyzing the response/output of the application (which often returns back to the caller the DBMS error response unfiltered): a typical error resulting from an injection would look like: ***“You have an error in your SQL syntax. Check your manual for the right syntax to use near FROM db_users.user_table”.***

SQL Injection Attack – Exploit



14

- 3. **Exploit:** perform the attack through malicious SQL input injection.
- SQL Injection Attacks can be further classified in:
 - ▣ First Order SQL Injection Attack
 - ▣ Second Order SQL Injection Attack

Scenario



15

- **Example 1:** Incorrectly filtered escape characters (like single quote)

SQL statement := "SELECT * FROM users
WHERE name = '\" + userName + \"'; "

- If a malicious user puts in the username variable the following string:

a' or 't'='t

- Then the following SQL statement is generated:

*SELECT * FROM users WHERE name = 'a'
OR 't'='t';*

Scenario



16

- Example 2: in this case the SQL engine allows the execution of multiple commands in the same SQL query

```
SQL statement := "SELECT * FROM users  
WHERE name = '" + userName + "'; "
```

- If a malicious user puts in the *username* variable the following string:

```
a';DROP TABLE users; SELECT * FROM data  
WHERE name LIKE '%'
```

- Then the following SQL statement is generated:

```
SELECT * FROM users WHERE name =  
'a';DROP TABLE users; SELECT * FROM  
DATA WHERE name LIKE '%';
```


Scenario



17

- Example 3: a user supplied field is not strongly typed or is not checked for type constraints by the application

```
SQL statement := "SELECT * FROM data  
WHERE id = " + a_variable + ";"
```

- With this statement the developer intended a_variable to be a number correlating to the "id" field. However, if it is in fact a string, the end users may manipulate the statement as they choose, thereby bypassing the need for escape characters
- For example, if the malicious input is injected:

```
1; DROP TABLE users
```

- Then the following SQL statement is generated:

```
SELECT * FROM DATA WHERE id = 1; DROP  
TABLE users;
```

Scenario



18

- Even if an application always escapes single - quotes, an attacker can still inject SQL as long as data in the database is re-used by the application
- An attacker might register with an application, creating a username
 - ▣ Username: `admin'--`
 - ▣ Password: `password`
 - ▣ Suppose that the application correctly escapes the single quote, resulting in an 'insert' statement like this:
 - ▣ `insert into users values( 123, 'admin"--', 'password', 0xffff )`

Scenario



19

- Suppose that the application allows users to change their password. The ASP script code first ensures that the user has the 'old' password correct before setting the new password. The code might look like this:

```
username = escape( Request.form("username") );
oldpassword = escape( Request.form("oldpassword") );
newpassword = escape( Request.form("newpassword") );
var rso = Server.CreateObject("ADODB.Recordset");
var sql = "select * from users where username = '" +
    username + "' and password = '" + oldpassword +
    "'";
rso.open( sql, cn );
if (rso.EOF)
{
```

...

rso("username") is the username retrieved from the 'login' query



Scenario

20

- The query to set the new password might look like this:
 - ▣ `sql = "update users set password = '' + newpassword + '' where username = '' + rso("username") + ''"`
- Given the username `admin'--`, the query produces the following query:
 - ▣ `update users set password = 'password' where username = 'admin'--'`
- ***The attacker can set the admin password to the value of his choice, by registering as a user called admin'--.***

Note that the **double hyphens (--)** are used in SQL to denote comments. Everything after the -- until the end of the line is considered a comment not part of the SQL

SQL Injection Countermeasures



21

- Requirement definition phase:
 - ▣ A non-SQL style database which is not subject to these flaws may be chosen
- Design phase:
 - ▣ Duplicate any filtering done on the client-side on the server side
 - ▣ Follow the principle of least privilege when creating user accounts to a SQL database. Users should only have the minimum privileges necessary to use their account. If the requirements of the system indicate that a user can read and modify their own data, then limit their privileges so they cannot read/write others' data

SQL Injection Countermeasures



22

□ Implementation phase:

- ▣ Disallow meta-characters rather than escaping them
- ▣ Narrowly define the set of safe characters based on the expected value of the parameter in the request
- ▣ White-list style checking on any user input that may be used in a SQL command
- ▣ Build the SQL query strings using parameterized statements that bind variables. Parameterized statements that do not bind variables can be vulnerable to attack

SQL Injection Countermeasures



23

- Input validation
 - ▣ Complex matter
 - ▣ Often overlooked in the implementation phase since it does not add functionalities
- Approaches to input validation:
 - ▣ Transform (“filtering”) the input
 - ▣ *Blacklisting*: reject input that is known to be “bad”, **BUT**
 - Known “bad input” may change over time as new attack techniques develop
 - ▣ *Whitelisting*: accept only input that is known to be “good”

SQL Injection Countermeasures

Filtering Characters



24

- Input validation – filtering characters
 - ▣ Filter out characters like:
 - single quote ‘, double quote “,
 - Slash /, back slash \,
 - semi colon ;,
 - extended character like **NULL**,
 - carriage return,
 - new line, etc,
 - ▣ in all strings from:
 - input from users
 - parameters from URL
 - values from cookie

SQL Injection Countermeasures

Reject “bad” Input



25

- The application must act deterministically when it receives invalid characters from a user.
- For example, applications should patently reject users who submit two dash characters (--) or a semi-colon character (;) as part of their login name or password, and a high severity alert should be sent to application administrators.
- However, this somewhat harsh response may not be appropriate when an application receives a single apostrophe character as part of a person name (e.g., O'brien) or street address that is submitted by an authenticated user who is entering package shipping information. Nonetheless, the application should behave as intended, and a notification should be sent to application administrators if the submitted apostrophe was not handled properly by the application.
- Invalid characters should also consider numeric parameters

SQL Injection Countermeasures

Reject “bad” Input



26

Example: disallow select/insert/update/delete/drop/--/' in the user input

```
function validate_string( input )
    known_bad = array( "select", "insert",
        "update", "delete", "drop", "--", "'" )
    validate_string = true
    for i = lbound(known_bad) to ubound(known_bad)
        if (instr( 1, input, known_bad(i),
            vbtextcompare) <> 0 ) then
            validate_string = false
            exit function
        end if
    next
end function
```

SQL Injection Countermeasures

Reject “bad” Input



27

- Problem: when combining the filtering of data with validation of character sequences the attacker could still evade the detection
- Example:
 - A known bad input filter detects '--', 'select' and 'union' followed by a filter that removes single-quotes
 - The attacker could specify an input:
uni'on sel'ect @@version-'-
 - Since the single-quote is removed after the 'known bad' filter is applied, the attacker can simply intersperse single quotes in his known-bad strings to evade detection

SQL Injection Countermeasures

Accept only “good” input



28

Accept only input that is known to be “good”

- ▣ Narrowly define the set of safe characters based on the expected value of the parameter in the request
- ▣ Example:

```
good_password_chars =
```

```
"abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ  
RSTUVWXYZ0123456789"
```

SQL Injection Countermeasures



Accept only “good” input

29

```
function validatepassword( input )
good_password_chars =
  "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ
  VWXYZ0123456789"

validatepassword = true
for i = 1 to len(input)
  c = mid(input, i, 1)
  if (InStr(good_password_chars, c) = 0)
    then
      validatepassword = false
      exit function
  end if
next
end function
```

SQL Injection Countermeasures

Accept only “good” input



30

- Good character sets can implicitly defined through *regular expressions*
- *RegExp* – pattern to match strings against
- Ex: suppose you have a *month* parameter which must be non-negative integer
 - ▣ RegExp: `^[0-9]*$` - 0 or more digits, safe subset
 - ▣ The `^`, `$` match beginning and end of string
 - ▣ `[0-9]` matches a digit, `*` specifies 0 or more

SQL Injection Countermeasures

Parameterized statements



31

- Parameterized statements use parameters (sometimes called placeholders or bind variables) instead of directly embedding user input in the SQL statement
- In many cases, the SQL statement is fixed. The user input is then assigned (bound) to a parameter
- An example using Java and the JDBC API:
 - `PreparedStatement prep = conn.prepareStatement("SELECT * FROM USERS WHERE PASSWORD=?"); prep.setString(1, pwd);`
- The reason why parameterized statements offer protection is that the query is parsed before the values of the input parameters are provided

SQL Injection Countermeasures

SQL Stored Procedures



32

- Using stored procedures (system or user-defined) perse does not guarantee against SQL injection if they do not validate the user input
- Moreover, stored procedures are often written in special scripting languages, they can contain other types of vulnerabilities, such as buffer overflows



QUESTIONS And Answers

